

KV260 平台 SmolVLM2 模型 FPGA 加速与 Decode 优化技术报告

测试报告与技术文档整理版

2026 年 6 月 15 日

摘要

本系统面向 SmolVLM2-500M 的多模态推理，在 KV260 上实现以 W8A8 为核心的算子级 FPGA 加速。设计并未将完整 transformer block 整体迁移至 PL，而是优先选择矩阵密集、形状稳定、融合收益高且精度风险可控的算子，包括 text tower 的 QKV fused、attention output projection、FFN fused，vision tower 的 QKV fused、linear、vision MLP fused，以及可选 decode FFN fused。

PL 端采用 RTL PE 阵列与 HLS 控制/搬运相结合的混合架构。RTL PE 负责高并行 INT8 MAC，HLS 负责任务解析、activation/weight 读取、bias、scale/requant、FFN 激活函数融合和输出写回。Host 侧通过 512 MiB u-dma-buf arena 与 PL 交换权重、激活、descriptor 和输出，并通过 cached mmap、A8/POT 快速量化、profile 工具链降低运行时开销。

最终提交配置在 OCRBench 上达到 17/30，prefill throughput 为 35.78 tokens/s，decode throughput 为 7.28 tokens/s，平均功耗 5.79 W，能效为 1.22 tokens/J。软件侧进一步针对长 decode 场景引入 sparse attention，以减少历史 KV cache 的访问和 CPU attention 计算压力。

1 设计目标与系统边界

1.1 加速目标

SmolVLM2-500M 的主要矩阵形状相对固定，适合采用专用化数据通路。系统优化优先覆盖 prefill 与 vision encoder 中的大 batch 矩阵，同时在 decode 阶段只保留收益明确、风险可控的 FFN 路径。

表 1: 主要加速模块与典型矩阵形状

模块	典型形状
text QKV	$m \times 960 \rightarrow 1600$
text O projection	$m \times 960 \rightarrow 960$
text FFN	$960 \rightarrow 2560 \rightarrow 960$
vision QKV	$m \times 768 \rightarrow 2304$
vision O / linear	$m \times 768 \rightarrow 768$
vision MLP	$768 \rightarrow 3072 \rightarrow 768$

其中 m 在实际请求中常见为 7、64、430 或 1024。大 batch 路径更适合摊销权重读取、任务提交和 DMA 同步成本；小 batch decode 则需要谨慎评估固定启动开销和精度回归风险。

1.2 实现边界

当前完成版本不在 PL 端实现 attention softmax、RoPE 或完整 transformer block。attention core 仍由 CPU 执行，PL 负责 attention 前后的线性投影与 FFN。该边界有三点考虑：

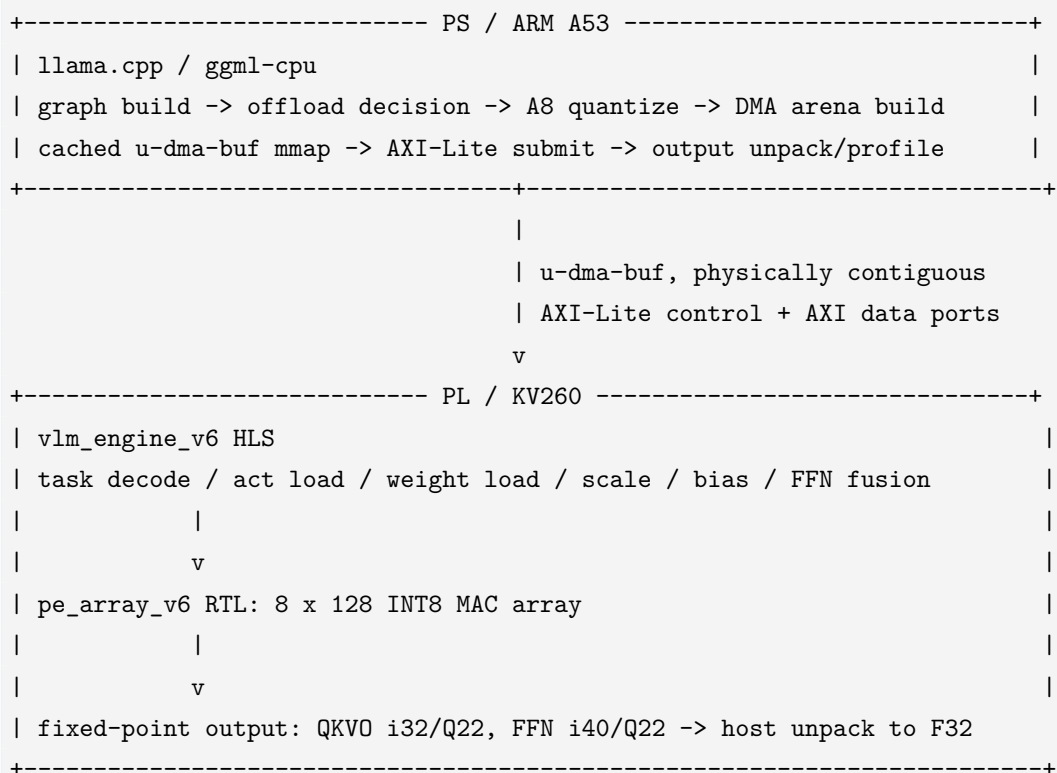
- **资源可控：**KV260 的 DSP 与 BRAM 资源紧张，完整 block 上板会显著增加资源、时序和验证压力。
- **精度可回归：**线性层与 FFN 更容易进行算子级对齐、profile 和逐项回归。
- **比赛版本稳定：**优先交付已稳定收敛的算子级部署路径，避免未完成模块影响最终结果。

2 总体架构

2.1 系统视图

系统采用 PS 侧运行时调度与 PL 侧专用矩阵阵列协同的结构。PS 侧完成 llama.cpp 图构建、offload 决策、激活量化、DMA arena 构造、AXI-Lite 提交和输出解包；PL 侧由 HLS 顶层解析任务并驱动 RTL PE 阵列完成 INT8 GEMM。

Listing 1: Host-PL 协同结构示意图



核心原则是“复杂控制用 HLS，主矩阵计算用 RTL”。HLS 便于维护多算子任务和 ABI；RTL PE 阵列则保证 MAC 结构可控、资源可预期、时序可分析。

2.2 数据格式

表 2: 系统数据格式

数据	格式	说明
权重	W8 / POT group128	GGUF Q8_0 权重在 host 侧预打包为硬件布局
激活	A8 / POT group128	host 侧将 F32 激活量化后提交给 PL
QKV/O 输出	i32/Q22	降低 output unpack 与读回带宽
FFN 输出	i40/Q22 in int64 slot	保持 FFN 精度基线
host tensor	F32	与 llama.cpp 后续 CPU op 兼容

QKVO 输出压缩为 i32/Q22 后，输出读回数据量约减半；FFN 继续使用 i40/Q22，以降低激活函数近似与累加量化带来的精度风险。

3 PL 端算子映射

3.1 RTL PE 阵列

pe_array_v6 使用 8 行 $\times$ 128 列的 INT8 MAC 阵列，每拍接收 8 个 activation 与 128 个 weight，对 8×128 个输出元素并行累加。N 维 tile 取 128，既适配 decode 的列并行需求，也能覆盖 prefill 大矩阵。

表 3: PE 阵列关键参数

项目	数值
PE 数量	1024
行并行	8 rows
列并行	128 columns
A stream	64 bit/cycle
W stream	2×512 bit/cycle
Psum stream	256 bit/beat
DSP 使用	约 1024 DSP for PE

3.2 HLS Engine

vlm_engine_v6 负责 descriptor 解析、激活与权重读取、bias 与 scale/requant、FFN 融合、结果写回，并向 RTL PE 发送 A/W/control stream、接收 psum stream。顶层接口包含四路 AXI master：

表 4: HLS 顶层 AXI 接口

端口	用途
gmem_w0	weight / scale columns 0..63
gmem_w1	weight / scale columns 64..127
gmem_act	activation / task / bias
gmem_out	fixed-point output / intermediate context

3.3 算子融合策略

QKV fused. Q、K、V 三个 projection 共用同一份输入 activation。Host 侧将三组权重组织为 multi-output task，PL 侧连续生成 Q/K/V 输出，从而减少重复 activation 处理和提交开销。

FFN fused. text FFN 在 PL 内融合 gate、up、SiLU 与 down projection:

Listing 2: Text FFN fused 数据流

```
x
-> gate = x * W_gate
-> up   = x * W_up
-> mid  = SiLU(gate) * up
-> down = mid * W_down
```

中间张量 mid 不回写 DDR，而是在 PL 内完成定点激活、requant 和 down projection。vision MLP 使用 GELU+bias fused 路径，同样避免完整中间张量落 DDR。

Bias fusion. dense/O/QKV 的 bias 初始化融合进第一次 K-tile absorb:

Listing 3: Bias 融合逻辑

```
kt == 0: out_acc = bias + scaled_psum
kt > 0: out_acc += scaled_psum
```

该策略减少一次独立 bias init pass，也降低小矩阵任务的固定启动成本。

3.4 Weight Ping-Pong 与 Dataflow

PL 端按 K tile 调度权重。优化后将 weight load 与 compute/absorb 解耦:

Listing 4: Weight ping-pong 调度

```
load bank0
compute bank0 while prefetch bank1
compute bank1 while prefetch bank0
...
```

跨 dataflow process 只传 stream token，weight buffer 所有权保留在 compute-owner 内部，避免 HLS 将跨进程数组降成 PIPO/FIFO 后导致 slab 复用失败。BM4 prefill 路径下，同一个 K tile 权重可复用于 4 个 8-row slab，提高权重读取摊销效率。

3.5 输出格式优化

表 5: 输出格式与优化目的

输出类型	格式	目的
QKV/O	i32/Q22	输出读回带宽减半，unpack 更快
FFN/vision MLP	i40/Q22	保持精度稳定

Host 侧根据 task type 和 row stride 选择 i32 或 i64 unpack，DMA sync 也使用真实输出范围，避免无效 cache maintenance。

4 Host-PL 协同与运行时优化

4.1 llama.cpp 接入

系统在 ggml-cpu 后端接入 Zynq runtime。普通 linear 通过 `mul_mat` 路径拦截，FFN 通过 custom op 插入。offload 判断同时检查 tensor type、shape、tensor name、row count 以及 v6/v5 runtime 开关。不满足条件的 op 直接回到 CPU 路径，保证通用性和稳定性。

4.2 DMA Arena 与 Cached mmap

`setup_udmabuf.sh` 创建 512 MiB 连续 DMA buffer，runtime 从 sysfs 读取 size 与 `phys_addr`。Host 侧通过 mmap 写入 task、activation、weight 和 bias，并将物理地址写入 AXI-Lite control register；PL 计算完成后，Host 从同一 arena 读取 fixed-point 输出并转回 F32。

启用 `GGML_ZYNQ_UDMABUF_CACHED=1` 后，ARM 读取 PL 输出时走 cacheable mapping，同时 runtime 保留 `sync_for_device` 与 `sync_for_cpu` 以保证一致性。

表 6: Cached mmap 对 Host-PL 协同开销的影响

配置	FPGA profile	Output Copy
未 cached	15318.41 ms	8282.68 ms
cached mmap	7928.60 ms	891.31 ms

cached mmap 将 output readback 降低约 9.3 倍，是 Host-PL 协同中最关键的软件优化之一。

4.3 A8 激活量化优化

PL 输入采用 A8/POT group128。Host 侧量化路径主要包含两个优化开关：`GGML_ZYNQ_A8_FAST_POT=1` 使用 POT 等价缩放替代逐元素除法；`GGML_ZYNQ_A8_NEON=1` 使用 AArch64 NEON 完成 `max_abs`、`round`、`clamp` 与 `narrow`。

表 7: A8 激活量化优化效果

指标	优化前	FAST_POT + NEON
Activation Quantize	2540.23 ms	476.47 ms
FPGA profile total	9072.18 ms	7008.37 ms

量化耗时降低约 81.2%。

5 Decode 阶段 CPU 侧优化

5.1 瓶颈来源

在原始实现中，decode 每生成一个 token 都需要对已有上下文中的 KV cache 执行 attention 计算。随着输出长度增加，单步 decode 的 attention 开销近似线性增长，既包含点积、softmax 和加权求和，也包含对 KV cache 的连续访存。对于 KV260 上的 Cortex-A53 CPU，这部分计算和访存都较为敏感。

因此，软件侧优化优先减少 decode attention 的实际工作量，而不是继续在短序列路径或局部小算子上做微小替换。后续尝试表明，RoPE cache、fast exp、单独 GEMV 小内核等局部优化对端到端 decode 吞吐贡献有限。

5.2 Sparse Attention 方法

最终保留的软件优化路线是 sparse attention。该方法在 decode 阶段不再完整访问所有历史 KV，而是选择性保留对输出质量更重要的 token：

- 保留序列开头的若干 token，以维持系统提示、图文任务说明和全局语义；
- 保留最近一段 token，以保证局部上下文连续；
- 对中间较长历史 token 按固定步长稀疏采样，以降低访存与计算量。

参与 attention 的历史集合可表示为：

$$KV_{\text{sparse}} = KV_{\text{head}} \cup KV_{\text{recent}} \cup KV_{\text{stride}}$$

其中， KV_{head} 表示开头保留区域， KV_{recent} 表示最近上下文窗口， KV_{stride} 表示中间区域的稀疏采样结果。该设计参考了长上下文推理中保留初始 token 与最近窗口、结合局部窗口与全局 token 的稀疏访问思路。

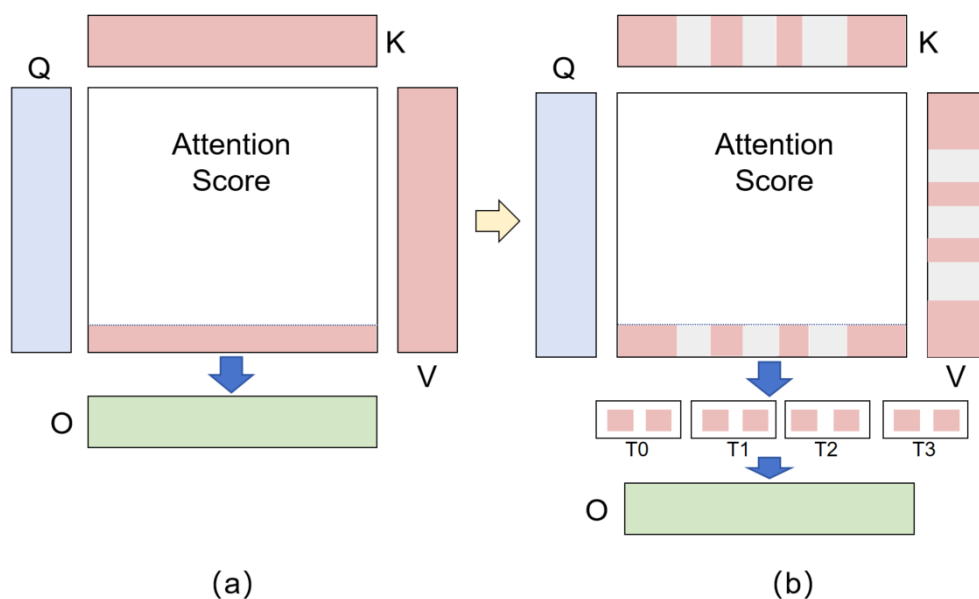


图 1: Decode 阶段 Sparse Attention 优化示意图

图 1 中，dense attention 会对完整历史 K/V cache 计算 attention score；sparse attention 仅计算被保留的 K/V block，并将其均衡分配到多个 CPU 线程上执行。蓝色表示当前 query，红色表示参与本次 decode step 的 K/V block 及 score 片段，灰色表示被跳过的历史 K/V block，绿色表示最终输出向量。

5.3 工程实现要点

动态触发与路径选择. 为避免 sparse attention 在短上下文或精度敏感样本中产生不必要影响，实现中加入最小上下文长度阈值。只有当 KV 长度超过阈值时才启用 sparse attention，否则仍走原始 dense attention 路径。这样可以使 OCRBench 中大量短答案样本基本保持原始计算路径，而吞吐量测试中的长 decode 场景能够触发优化。

Split-KV 路径适配. 调试过程中发现，llama.cpp 在长上下文 decode 中会走 split-KV partials 路径。如果 sparse attention 只覆盖普通 attention 分支，就无法真正作用于主要热点。因此软件实现需要适配 split-KV 情况，使稀疏后的 KV 选择进入实际 decode 主路径。该问题修正后，sparse attention 才稳定产生正向收益。

多线程负载均衡. 稀疏采样后，保留下来的 KV block 不再是简单连续区间。如果仍按原始区间划分线程任务，容易出现线程负载不均，部分线程提前结束并等待其他线程。balanced sparse attention 调度将保留的 KV block 尽量均匀分配到多个 CPU 线程，减少同步等待，提高 ARM 四核利用率。

5.4 方案筛选结论

除 sparse attention 外，软件侧还尝试过多种优化方向。KV cache 量化具备一定速度潜力，但准确率风险更高；RoPE 预计算和 fast exp 主要优化局部算子，端到端收益不明显；手写 decode GEMV/attention 小内核难以稳定超过 llama.cpp 原有 ARM NEON 路径；prompt lookup speculative decoding 在当前任务中接受率较低。因此，最终保留 sparse attention 作为主优化路径，其余方法作为探索记录，不纳入默认提交配置。

6 测试数据与结果

表 8: 平台配置概览

项目	内容
硬件平台	Xilinx KV260 / Zynq UltraScale+
模型	SmolVLM2-500M-Video-Instruct Q8_0
主要优化	W8A8 算子级 FPGA 加速、Host-PL 数据通路优化、decode sparse attention
默认边界	attention softmax、RoPE 等仍保留在 CPU 侧

6.1 最终提交指标

表 9: 最终提交配置评测结果

指标	结果
OCRBench sample-30	17 / 30
Prefill throughput	35.78 tokens/s
Decode throughput	7.28 tokens/s
Average power	5.79 W
Tokens/J	1.22

6.2 End-to-End Profile

表 10: server_profile_v12.log 端到端指标

指标	数值
prompt total tokens	501
prompt cached tokens	0
prompt eval tokens	501
decode tokens	119
CPU events	35385
FPGA events	11650
prompt eval	15839.93 ms
decode eval	28678.08 ms
total time	44518.01 ms

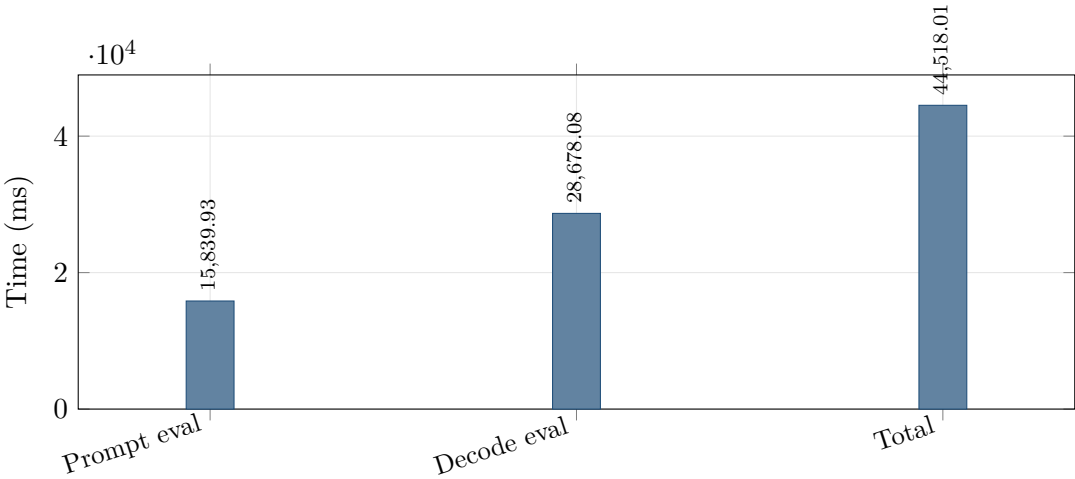


图 2: server_profile_v12 端到端耗时对比

表 11: CPU phase summary

Domain	Phase / Chunk	Tokens	Graph	Top Stage	Top Share
vision	prefill / vision	64	7922.26 ms	attn_core	66.47%
text	prefill / embedding	64	818.35 ms	ffn_fused	31.09%
text	prefill / token	7	160.17 ms	ffn_fused	49.55%
text	prefill / token	430	6725.72 ms	attn_core	50.59%
text	prefill / token	1	4.24 ms	sampler	100.00%
text	decode / token	1	27235.76 ms	ffn_fused	31.18%

表 12: FPGA runtime breakdown

Component	Time	Share
Hardware Wait	11332.19 ms	78.09%
Output Readback / Copy	1226.60 ms	8.45%
Activation Quantize	703.53 ms	4.85%
Submission Build	266.06 ms	1.83%
DMA Sync To Device	245.85 ms	1.69%
DMA Sync To CPU	83.78 ms	0.58%
MMIO Submit	66.12 ms	0.46%
Weight Cache / Pack	46.43 ms	0.32%
CPU Fallback	0.00 ms	0.00%

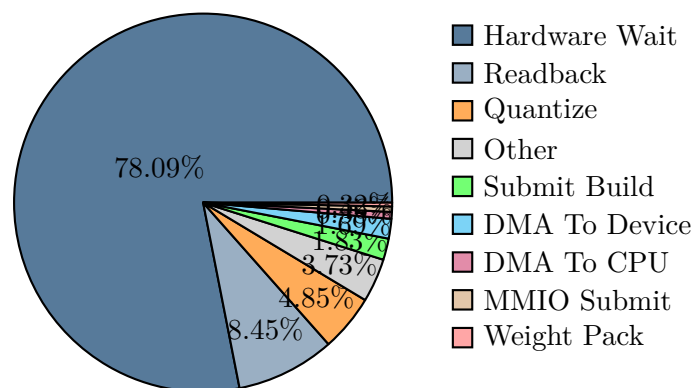


图 3: FPGA runtime breakdown 占比

FPGA profile 总耗时为 14510.99 ms, 其中 Hardware Wait 占 78.09%, 仍是最主要瓶颈; Output Readback / Copy 与 Activation Quantize 分别占 8.45% 和 4.85%。这说明后续优化应继

续围绕 PL 计算等待、权重读取摊销和小 batch decode 路径展开，同时保留 cached mmap 与 A8 快速量化等已验证的软件优化。

6.3 Hot Ops

表 13: 主要 PL 算子 profile

算子	Shape	Count	Avg	HW
vision MLP fused	m=1024 n=768 k=768	12	87.94 ms	71.65 ms
vision QKV fused	m=1024 n=2304 k=768	12	33.65 ms	16.97 ms
vision linear-bias	m=1024 n=768 k=768	12	15.76 ms	6.22 ms
text FFN fused	m=430 n=960 k=2560	31	45.57 ms	36.96 ms
text QKV fused	m=430 n=1600 k=960	32	13.57 ms	6.72 ms
text O projection	m=430 n=960 k=960	32	9.72 ms	4.04 ms
decode FFN fused	m=1 n=960 k=2560	3777	1.76 ms	1.61 ms
decode QKV fused	m=1 n=1600 k=960	3776	0.51 ms	0.35 ms
decode GEMV	m=1 n=960 k=960	3776	0.41 ms	0.27 ms

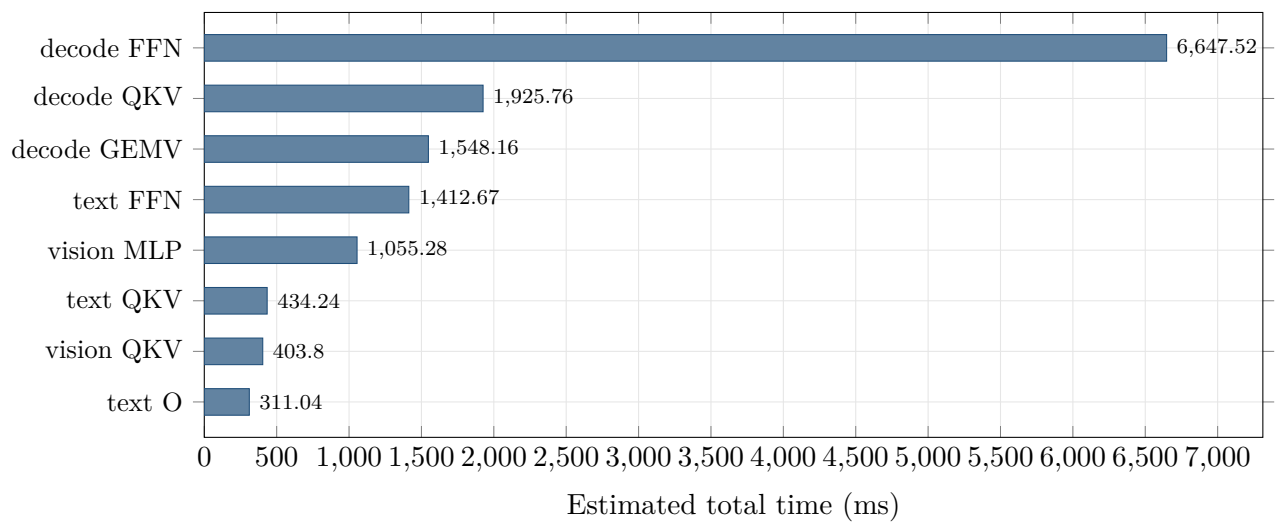


图 4: Hot Ops 总耗时估计 ($\text{Avg} \times \text{Count}$)

6.4 资源与时序

表 14: 资源使用

资源	使用	板卡上限	比例
CLB LUT	42007	117120	35.9%
CLB FF	86233	234240	36.8%
BRAM Tile	123.5	144	85.8%
URAM	16	64	25.0%
DSP	1069	1248	85.7%

300 MHz routed timing 中, WNS 为 -0.710 ns, TNS 为 -28002.639 ns。该版本主要受 DSP 与 BRAM 约束; 后续版本虽通过 BRAM/URAM 迁移和 cache 合并降低过 BRAM 占用, 但 full timing closure 仍需继续优化 RTL PE fanout、psum drain 与 HLS control path。

7 部署与评测流程

```
cd ~/aicas

# 1. 编译 llama.cpp
mkdir -p ./output
cmake -S llama.cpp-vendor -B build-zynq \
  -DLLAMA_PROFILE_FINE=OFF \
  -DGGML_ZYNQ_VLM=ON \
  -DGGML_OPENMP=ON \
  -DLLAMA_BUILD_SERVER=ON \
  -DLLAMA_BUILD_TOOLS=ON \
  -DLLAMA_BUILD_TESTS=OFF \
  -DLLAMA_CURL=OFF
cmake --build build-zynq --target llama-server -j4

# 2. 设置pl端频率(300MHz)
echo 300000000 | sudo tee /sys/devices/platform/fclk0/set_rate
sudo cat /sys/kernel/debug/clk/clk_summary | grep -Ei 'pl0|fclk|fpga'

# 3. 设置环境变量,加载bitstream
source ./set_env.sh
chmod +x load_bit.sh
sudo -E bash load_bit.sh output/gemm.bit

# 4. 配置udmabuf
```

```
chmod +x setup_udmabuf.sh
bash ./setup_udmabuf.sh
```

5. 启动llama.cpp server,并等待启动成功

```
nohup sudo -E bash ./run.sh server > "./output/server.stdout.log" 2>&1 &
SERVER_PID=$!
echo $SERVER_PID > "$LOG_DIR/server.pid"
python3 eval/wait_for_server.py --host "$SERVER_HOST" --port "$SERVER_PORT" --pid-file "
    $LOG_DIR/server.pid" --timeout 600
```

6. 启动评测

```
source .venv/bin/activate
python3 eval/acc_eval.py --image_folder "$EVAL_OCR_IMAGE_DIR" --OCRBench_file "eval/
    sample/sample_30_org18.json" --output "$EVAL_OUTPUT_DIR/acc_eval_results.json"
python3 eval/throughput_eval.py --image "$EVAL_IMAGE_PATH" --output "$EVAL_OUTPUT_DIR/
    throughput_metrics.json"
python3 eval/energy_eval.py --image "$EVAL_IMAGE_PATH" --output "$EVAL_OUTPUT_DIR/
    energy_metrics.json" --sample_hz 100
python3 eval/ttft_eval_multiprompt.py --config "$EVAL_TTFT_CONFIG" --output "
    $EVAL_OUTPUT_DIR/ttft_eval_results.json"
python3 eval/merge_results.py \
    --acc "$EVAL_OUTPUT_DIR/acc_eval_results.json" \
    --throughput "$EVAL_OUTPUT_DIR/throughput_metrics.json" \
    --energy "$EVAL_OUTPUT_DIR/energy_metrics.json" \
    --ttft "$EVAL_OUTPUT_DIR/ttft_eval_results.json" \
    --output "$EVAL_OUTPUT_DIR/aicas_submission.json"
```

8 结论

本队伍的实现路线可以概括为“算子级融合 + W8A8 专用矩阵阵列 + Host-PL 数据通路优化”。相比完整 transformer block 上板,这条路线更稳、更容易进行精度回归,也更适合 KV260 的资源边界。

已经验证有效的关键点包括:

- 8×128 RTL PE 阵列承担主 INT8 GEMM;
- HLS 统一调度 dense/QKV/O/FFN/vision MLP;
- FFN 与 vision MLP 中间结果不落 DDR;
- QKVO i32/Q22 输出降低读回成本;
- cached u-dma-buf 将 output readback 降低约 9 倍;
- A8 FAST_POT + NEON 将激活量化降低约 81%;
- profile 工具能够分解 hardware wait、quant、DMA、pack、submit 等瓶颈;

- decode sparse attention 在长上下文场景中减少冗余 KV 访问，并在准确率未明显下降的前提下改善吞吐。